

Kazakh-British Technical University

Data Collection and Preparation

STUDENTS' INDEPENDENT STUDY (SIS) №1

Do UFOs Only Show Up When It's Cloudy?

Full Name:

Tuimebayev Doszhan 23B030405,

Shoibek Arman 23B030412,

Akhmetova Kamila 23B030786,

Zinatov Ruslan 24B031807

Part: 1 — Data Collection (API)

08.10.2026

Almaty

Part 1. Data Collection: Web Scraping

Data source: Wikipedia — List of reported UFO sightings

(https://en.wikipedia.org/wiki/List_of_reported_UFO_sightings). The page has several tables with notable UFO sightings from ancient times to today. Every table has four columns: Date, Name, Location and Description.

Tools: Python, requests, BeautifulSoup, urllib.robotparser, pandas (Google Colab).

Approach

1. Checked robots.txt of Wikipedia with RobotFileParser. The result can_fetch = True, so scraping of this page is allowed. A User-Agent header was added to all requests.
2. Downloaded the page with requests.get (status code 200) and parsed the HTML with BeautifulSoup.
3. Found all tables with class "wikitable" – 7 tables, one for each period (ancient times, 20th century, 21st century etc.).
4. Went through every row (tr) of every table and took the text of the first four cells (td). Reference marks like [12] (sup tags) were removed. For each table the nearest heading (h2/h3) was saved as the period column.
5. Created a DataFrame with columns date, name, location, description, period and saved it to ufo_raw.csv.

```
Part 1. Data Collection: Web Scraping

[ ] ▶ import requests
import pandas as pd
from bs4 import BeautifulSoup
from urllib.robotparser import RobotFileParser
import numpy as np

h = {"User-Agent": "Mozilla/5.0 (KBTU student project)"}
u = "https://en.wikipedia.org/wiki/List_of_reported_UFO_sightings"

[ ] r = requests.get("https://en.wikipedia.org/robots.txt", headers=h)
print(r.status_code)
rp = RobotFileParser()
rp.parse(r.text.splitlines())
print("can fetch:", rp.can_fetch("*", u))

200
can fetch: True

[ ] r = requests.get(u, headers=h)
print(r.status_code)
s = BeautifulSoup(r.text, "html.parser")
print(s.title.text)

200
List of reported UFO sightings - Wikipedia

[ ] t = s.find_all("table", class_="wikitable")
print(len(t))
print([th.get_text(strip=True) for th in t[0].find_all("th")])

7
['Date', 'Name', 'Location', 'Description']

[ ] rows = []
for x in t:
    p = x.find_previous(["h2", "h3"])
    p = p.get_text(strip=True) if p else None
    for tr in x.find_all("tr")[1:]:
        td = tr.find_all("td")
        if len(td) < 4:
            continue
        for sup in tr.find_all("sup"):
            sup.decompose()
        rows.append([c.get_text(" ", strip=True) for c in td[:4]] + [p])
print(len(rows))

137
```

Results

Final table: 137 rows, 5 columns

Columns: date, name, location, description, period – all text (object)

Missing values: 0

Rows with full date (YYYY-MM-DD): 104, rows without exact day (like "c. 1947", "1963", "218 BC"): 22

```
[ ] df.info()
print(df.isna().sum())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 137 entries, 0 to 136
Data columns (total 5 columns):
#   Column      Non-Null Count  Dtype
---  ---
0    date        137 non-null    object
1    name         137 non-null    object
2    location     137 non-null    object
3    description  137 non-null    object
4    period       137 non-null    object
dtypes: object(5)
memory usage: 5.5+ KB
date      0
name      0
location  0
description 0
period    0
dtype: int64
```

Problems found in the raw data: some dates are ranges ("1952-07-12 to 1952-07-29"), some have no exact day, some are before 1940 (no weather data in Open-Meteo), and some locations are not a real place ("Low Earth orbit", "Middle East"). Location is written as "Country; City, Region". These problems are solved in the cleaning part. The file ufo_raw.csv was passed to the next part.

Part 2. Data Collection: API

API: Open-Meteo — Historical Weather API (<https://archive-api.open-meteo.com/v1/archive>) and Geocoding API (<https://geocoding-api.open-meteo.com/v1/search>). Both are free and need no API key. The Historical Weather API returns daily weather for any point on Earth from 1940 until today.

Input: ufo_raw.csv from Part 1. Tools: Python, requests, pandas, time (Google Colab).

Approach

1. Loaded ufo_raw.csv (137 rows) and added an id column (row number), so the weather can be joined back to the sightings in the next part.
2. Took the exact date of every sighting. Spaces were removed ("1947- 07-08" → "1947-07-08"), the first YYYY-MM-DD was taken with a regular expression (for ranges like "1952-07-12 to 1952-07-29" the first day) and converted with `pd.to_datetime`. Weather data starts in 1940, so only sightings from 1940 with an exact day were kept – 103 rows.
3. Split the location "Continent Country ; Town, Region" into the country and a list of places. Words like "near" and "north of" were removed ("a farm near McMinnville" → "McMinnville").
4. Found coordinates with the Geocoding API (name = town, count = 10). From the results we took a town (feature_code PPL) with the same name and country, and with the same region if it was given. For the US the state must match, because many US towns have the same name (Exeter, California and Exeter, New Hampshire). 58 sightings got coordinates. 45 were skipped, because the location is a whole state or region ("Kentucky", "East Anglia", "Ukrainian war zone") or not a place on land ("Low Earth orbit").
5. For each of the 58 sightings sent a request to the Historical Weather API with latitude, longitude and dates from 15 days before to 15 days after the sighting. Daily variables: `cloud_cover_mean` (%), `precipitation_sum` (mm), `temperature_2m_mean` (°C). `time.sleep` was added between requests. All 58 requests returned status code 200.
6. From each response took the weather on the sighting day and the average over the 31 days (normal weather for that place and season), and saved the table to `weather_data.csv`.

Part 2. Data Collection: API

API: Open-Meteo — [Historical Weather API](#) and [Geocoding API](#). Free, no API key needed.

Input: `ufo_raw.csv` from Part 1. For every sighting we get the weather (cloud cover, precipitation, temperature) on that day and place.

```
(12)
import requests
import pandas as pd
import time

df = pd.read_csv('ufo_raw.csv')
df['date'] = df.index
print(df.shape)
```

(137, 6)

Step 1. Exact dates

The weather API needs an exact day, and its data starts in 1940. Dates look like 1947- 07-08 (with a space) or ranges like 1952- 07-12 to 1952- 07-29, so we remove spaces and take the first YYYY-MM-DD. Rows like c. 1947 or 218 BC have no exact day.

```
(13)
d = df['date'].str.replace(" ", "").str.extract(r"(\d{4})-(\d{2})-(\d{2})")[0]
df['d'] = pd.to_datetime(d, format="%Y-%m-%d", errors="coerce")
print("exact date:", df['d'].notna().sum())

w = df[df['d'].dt.year == 1940].copy()
print("exact date from 1940:", len(w))
```

exact date: 188
exact date from 1940: 183

Step 3. Coordinates (Geocoding API)

The weather API needs latitude and longitude. We search the first place (the town) and take a result that:

- is a town or city (`feature_code` starts with PPL).
- has the same name and is in the same country.
- is in the same region/state, if the region is given.

The US has many towns with the same name (Exeter is in California and in New Hampshire), so for the US the state must match. Rows with only a state or a region (Kentucky, East Anglia, Ukrainian war zone) are too big to have one weather value, so they are skipped.

```
(14)
g = "https://geocoding-api.open-meteo.com/v1/search"

def geocode(c, places):
    if len(places) == 0:
        return None, None, None
    q, rest = places[0], places[1:]
    r = requests.get(g, params={"name": q, "count": 10})
    found = []
    for x in r.json().get("results", []):
        xc = x.get("country", "")
        if not x.get("feature_code", "").startswith("PPL"):
            continue
        if x["name"] not in q and q not in x["name"]:
            continue
        if c != "" and (xc == "" or (c not in xc and xc not in c)):
            continue
        found.append(x)
    for x in found:
        adm = [x["name"], x.get("admin1", ""), x.get("admin2", ""), x.get("country", "")]
        if any(p in a for p in rest for a in adm):
            return x["name"], x["latitude"], x["longitude"]
    if found and c != "" and c != "United States":
        x = found[0]
        return x["name"], x["latitude"], x["longitude"]
    return None, None, None

print(geocode("United States", ("Exeter", "New Hampshire")))
```

('Exeter', 42.98148, -78.94783)

```
(14)
res = []
for c, p in zip(w['country'], w['places']):
    res.append(geocode(c, p))
    time.sleep(0.2)

w['place'], w['lat'], w['lon'] = zip(*res)
print("found:", w['lat'].notna().sum())
print("not found:", w['lat'].isna().sum())
w[w['lat'].isna()]['location'].head(10)
```

found: 58
not found: 45

Location

27	North America United States ; North of Mount R...
29	North America United States ; En route from Bo...
32	North America United States ; Puget Sound near...
34	North America United States ; Kentucky
35	North America United States ; New Mexico
37	North America United States ; North Dakota
38	North America United States ; Mount Palomar ob...
44	Asia Korea ; Wonsan and Sunchon
49	Europe Italy ; Stadio Artemio Franchi in Florence
51	Europe United Kingdom ; Suffolk, England

dtype: object

```
(17)
w = w[w['lat'].notna()]
w[['date', 'location', 'place', 'lat', 'lon']].head(10)
```

	date	location	place	lat	lon
23	1942-02-24	North America United States ; Los Angeles, Cal...	Los Angeles	34.05223	-118.24368
26	1946-05-18	Europe Sweden ; Angerholm, Kristianstads County	Angerholm	56.24280	12.86219
30	1947-07-07	North America United States ; Phoenix, Arizona	Phoenix	33.44838	-112.07404
31	1947-07-08	North America United States ; about 30 mi. nor...	Roseell	33.39437	-104.53491
36	1948-07-24	North America United States ; Montgomery, Alabama	Montgomery	32.36681	-86.29997
39	1950-03-15 to 1950-03-18	Farmington, New Mexico	Farmington	36.72806	-108.21869
40	1950-05-11	North America United States ; a farm near McM...	McMinnville	45.21012	-123.19872
41	1950-08-15	North America United States ; Great Falls, Mon...	Great Falls	47.50024	-111.30081
42	1951-01-20	North America United States ; Sioux City, Iowa	Sioux City	42.49999	-96.40031
43	1951-08-25	North America United States ; Lubbock, Texas	Lubbock	33.57786	-101.85517

Step 4. Weather (Historical Weather API)

One request example – Roswell, 1947-07-08. We ask for 31 days (15 days before and after the sighting), so later we can compare the sighting day with the normal weather in that place at that time of year.

Daily variables: cloud_cover_mean (%), precipitation_sum (mm), temperature_2m_mean (°C).

```
(14)
> a = "https://archive-api.open-meteo.com/api/archive"
> v = "cloud_cover_mean,precipitation_sum,temperature_2m_mean"

r = requests.get(a, params={"latitude": 33.39, "longitude": -104.52, "start_date": "1947-06-23",
                             "end_date": "1947-07-23", "daily": v, "timezone": "auto"})
j = r.json()
print(j["daily_units"])
pd.DataFrame(j["daily"]).head()

200
[{"time": "iso8601", "cloud_cover_mean": "%", "precipitation_sum": "mm", "temperature_2m_mean": "°C"}]

```

	time	cloud_cover_mean	precipitation_sum	temperature_2m_mean
0	1947-06-23	3	0.0	25.9
1	1947-06-24	57	6.0	24.7
2	1947-06-25	6	0.0	26.7
3	1947-06-26	24	0.0	30.0
4	1947-06-27	31	0.0	30.9

```
(15)
> out = []
> codes = []
> for x in w.iterrows():
>     a = x["d"] + pd.Timedelta(days=15).strftime("%Y-%m-%d")
>     e = x["d"] + pd.Timedelta(days=15).strftime("%Y-%m-%d")
>     r = requests.get(a, params={"latitude": x["lat"], "longitude": x["lon"], "start_date": s,
>                                 "end_date": e, "daily": v, "timezone": "auto"})
>     codes.append(r.status_code)
>     t = pd.DataFrame(r.json()["daily"])
>     day = t[t["time"] == x["d"]].strftime("%Y-%m-%d").iloc[0]
>     out.append({
>         "id": x["id"],
>         "date": x["d"].strftime("%Y-%m-%d"),
>         "place": x["place"],
>         "lat": x["lat"],
>         "lon": x["lon"],
>         "cloud_cover": day["cloud_cover_mean"],
>         "precipitation": day["precipitation_sum"],
>         "temperature": day["temperature_2m_mean"],
>         "cloud_cover_avg": round(t["cloud_cover_mean"].mean(), 1),
>         "precipitation_avg": round(t["precipitation_sum"].mean(), 2)
>     })
>     time.sleep(0.5)
> print(pd.Series(codes).value_counts())
> print(len(out))

200 58
Name: count, dtype: int64
58
```

Results

Final table: 58 rows, 10 columns

Columns: id, date, place, lat, lon, cloud_cover, precipitation, temperature, cloud_cover_avg, precipitation_avg

Data types: date and place are object, id and cloud_cover are int64, other columns are float64

Missing values: 0

```
cloud_cover, precipitation, temperature – weather on the sighting day. cloud_cover_avg, precipitation_avg –
average over the 31 days around it (normal weather for that place and season).
```

```
(16)
> wd = pd.DataFrame(out)
> print(wd.shape)
> wd.head(10)

(58, 10)

```

	id	date	place	lat	lon	cloud_cover	precipitation	temperature	cloud_cover_avg	precipitation_avg
0	23	1942-02-24	Los Angeles	34.05223	-118.24368	24	0.0	8.8	40.0	0.00
1	26	1948-09-18	Århus	56.24780	12.86219	91	4.3	11.6	51.2	1.24
2	30	1947-07-07	Phoenix	33.44838	-112.07404	1	0.0	34.4	17.8	0.03
3	31	1947-07-08	Roswell	33.39437	-104.52491	4	0.0	29.7	25.8	1.13
4	36	1948-07-24	Montgomery	32.36401	-86.29997	34	2.9	27.5	60.4	5.08
5	39	1950-03-15	Farmington	36.72806	-108.21869	38	0.0	4.5	43.8	0.53
6	40	1950-05-11	McMinnville	43.21012	-123.19672	13	0.0	15.6	58.6	1.53
7	41	1950-08-15	Great Falls	47.50024	-111.30081	41	0.1	21.6	43.0	0.91
8	42	1951-01-20	Sioux City	42.49999	-96.40031	100	4.4	-10.2	54.3	0.23
9	43	1951-08-25	Lubbock	33.57786	-101.85517	44	0.0	25.7	31.0	1.99

Далее: [Создать код с переменной wd](#) [New interactive sheet](#)

```
(17)
> print(wd.dtypes)
> print(wd.isna().sum())

id          int64
date        object
place       object
lat         float64
lon         float64
cloud_cover int64
precipitation float64
temperature float64
cloud_cover_avg float64
precipitation_avg float64
dtype: object

id          0
date        0
place       0
lat         0
lon         0
cloud_cover 0
precipitation 0
temperature 0
cloud_cover_avg 0
precipitation_avg 0
dtype: int64

1d is the row number in ufo_raw.csv, so in Part 3 the weather can be joined back to the sightings by 1d.
```

```
(18)
> wd.to_csv("weather_data.csv", index=False)
> print("saved:", wd.shape)

saved: (58, 10)
```

cloud_cover, precipitation and temperature are the weather on the sighting day; cloud_cover_avg and precipitation_avg are the averages for the 31 days around it, so it is possible to check if the sighting day was cloudier than usual. Weather was found for 58 of 137 sightings: the others are before 1940, have no exact day, or have only a region as location. The file weather_data.csv was passed to the next part to be merged with ufo_raw.csv by id.

Part 3. Data Collection

In this part our goal was to prepare both dataframes for merging – clean text, fix data types, remove missing values and duplicates, and normalize column names.

Approach:

1. Loaded ufo_raw.csv (137 rows, 5 columns) and weather_data.csv (58 rows, 10 columns). Added an id column to ufo (row number) so the weather can later be joined back to the sightings.
2. Column names: changed every column name to lower case and replaced spaces with _ (e.g. Cloud Cover → cloud_cover) in both dataframes.
3. Text cleaning: for the columns name, location, description, period, date:
 - a. converted to str type;
 - b. replaced any number of consecutive spaces/tabs with a single space;
 - c. removed leading and trailing spaces.
 - d. In the date column all inner spaces were removed ("1947- 07- 08" → "1947-07-08").
4. Missing values and duplicates:
 - a. Replaced empty strings and "nan" with NaN.
 - b. Checked for missing values: 0 in ufo, 0 in weather.
 - c. Checked for duplicates by (name, date): 0 duplicates found.
5. Data types in weather:
 - a. date → datetime64[ns];
 - b. cloud_cover, precipitation, temperature, cloud_cover_avg, precipitation_avg, lat, lon → numeric (int64 / float64) via pd.to_numeric(..., errors="coerce").
 - c. Dropped rows without date, cloud_cover or temperature (none found).

Part 3. Data cleaning

Here, ufo is our static web page (https://en.wikipedia.org/wiki/List_of_reported_UFO_sightings)

and w is our API (<https://open-meteo.com/en/docs/historical-weather-api>)

```
[42] ✓ Os
ufo = pd.read_csv("ufo_raw.csv")
w = pd.read_csv("weather_data.csv")
```

we will need id column for further task with merge

```
[43] ✓ Os
ufo = ufo.reset_index(drop=True)
ufo["id"] = ufo.index
```

- For each column name we change to lower case and replace space with _
- for each of 5 columns we change column type to str (replaces missing values to nan), for any number of consecutive spaces, tabs we replace them to one space
- As we did it previously with data cleaning, we are doing the same procedures and save them in ufo
- Checking

```
]
>S
ufo.columns = [c.strip().lower().replace(" ", "_") for c in ufo.columns]
w.columns = [c.strip().lower().replace(" ", "_") for c in w.columns]

for c in ["name", "location", "description", "period", "date"]:
    ufo[c] = ufo[c].astype(str).str.replace(r"\s+", " ", regex=True).str.strip()

ufo["date"] = ufo["date"].str.replace(r"\s+", "", regex=True)

print(ufo[["date", "name", "location"]].head())
print("dtypes:\n", ufo.dtypes)
```

```
***      date                                name \
0  c.1450BC    Thutmose III Jebel Barkal Stele
1      218BC                                Ships in the sky
2       76BC                Spark from a falling star
3       73BC    Flame-like wine-jars from the sky
4       65BC                                Fiery shield

                                location
0  Africa Ancient Egypt ; Jebel Barkal , Lower Egypt
1                                Europe Roman Republic ; Italia
2                                Asia Roman Republic ; Asia
3                                Asia Roman Republic ; Phrygia , Asia
4                                Europe Roman Republic ; Italia
dtypes:
date                object
name                object
location            object
description          object
period              object
id                  int64
dtype: object
```

Here, we are looking for nan, duplicates and empty strings and clean them if needed

we have 0 missing values and 0 duplicates

```
5] 0s ▶ ufo = ufo.replace({"": np.nan, "nan": np.nan})

print("Missing values before:\n", ufo.isna().sum())

before = len(ufo)
ufo = ufo.drop_duplicates(subset=["name", "date"], keep="first").reset_index(drop=True)
print(f"Duplicates removed: {before - len(ufo)}")

print("UFO after cleaning:", ufo.shape)
print(ufo.isna().sum())
```

```
... Missing values before:
  date      0
  name      0
  location  0
  description 0
  period    0
  id         0
dtype: int64
Duplicates removed: 0
UFO after cleaning: (137, 6)
  date      0
  name      0
  location  0
  description 0
```

- Here we are converting strings to datetime (or NaT)
- in a loop we are converting all numeric columns from strings to numbers
- we count missing values
- delete rows where we don't have date, cloudiness or temperature
- recounting shape

```
5] 0s w["date"] = pd.to_datetime(w["date"], errors="coerce")
for c in ["cloud_cover", "precipitation", "temperature",
          "cloud_cover_avg", "precipitation_avg", "lat", "lon"]:
    w[c] = pd.to_numeric(w[c], errors="coerce")

print("Missing values in weather before:\n", w.isna().sum())
w = w.dropna(subset=["date", "cloud_cover", "temperature"]).reset_index(drop=True)
print("weather after cleaning:", w.shape)
```

```
Missing values in weather before:
  id      0
  date    0
  place   0
  lat     0
  lon     0
  cloud_cover    0
  precipitation  0
  temperature    0
  cloud_cover_avg 0
  precipitation_avg 0
```



```

7] 0s print("=== UFO ==="); print(ufo.dtypes); print(ufo.shape)
    print("\n=== WEATHER ==="); print(w.dtypes); print(w.shape)

... === UFO ===
date            object
name            object
location        object
description      object
period          object
id              int64
dtype: object
(137, 6)

=== WEATHER ===
id              int64
date            datetime64[ns]
place           object
lat            float64
lon            float64
cloud_cover     int64
precipitation   float64
temperature     float64
cloud_cover_avg float64
precipitation_avg float64
dtype: object
(58, 10)

```

Results:

- ufo: 137 rows, 6 columns, 0 missing values, 0 duplicates.
- weather: 58 rows, 10 columns, 0 missing values.
- Both files are ready for logical merging.

Part 4. Merging

In this task our goal was to combine sightings (wikipedia) with the weather (open-meteo) logically, not just by row number. Every sighting should get the weather that was recorder on that same date and in that same place.

Approach:

1. Why id is not enough. The id column is just the row number in ufo_raw.csv. If the order of rows ever changed, the join would break. A correct merge must be based on real data — the date and the place of the sighting.
2. Preparing the keys.
 - a. Created a date_norm column in both dataframes: the date converted to a single string format YYYY-MM-DD using `pd.to_datetime(...).dt.strftime("%Y-%m-%d")`.
 - b. Created a place_key column: in ufo the town name was extracted from location (the part after ; and before the comma), in weather the place column was already clean from the Geocoding API. Both were stripped and lowercased.
3. Merging. Performed `ufo.merge(w, on=["date_norm", "place_key"], how="left")`. This means: for every row in ufo, find the row in weather with the same date and the same town. If a match exists, the weather is attached; if not, NaN is left.
4. Logical validation. After the merge we checked:
 - a. how many rows actually got weather (`cloud_cover.notna().sum()`);
 - b. whether the dates from ufo and weather coincide for the matched rows (this confirms the merge is logically correct, not accidental).
5. Added derived features based on the merged data:
 - a. has_weather — whether weather was found for the sighting;

- b. cloudy — cloud cover $\geq 50\%$;
 - c. cloudier_than_avg — cloud cover higher than the 31-day average;
 - d. rainy — precipitation > 0 ;
 - e. year — year of the sighting.
6. Aggregation by date. For each date we computed the number of sightings (sightings), the average cloud cover (avg_cloud) and the average temperature (avg_temp), and merged this aggregated table back into the main dataframe by date.

Results:

- Final dataframe df: 137 rows, 20 columns.
- Weather found for 58 rows out of 137 (the other 79 are events before 1940, events without an exact day, or events whose location is a whole region instead of a town).
- All 58 matched rows have the same date in ufo and weather (logical check passed).
- The cleaned dataframe was saved to ufo_clean.csv for the next part.

Part 4. Merging

1)

Os

```

ufo = ufo.copy()
ufo["date_norm"] = pd.to_datetime(ufo["date"], errors="coerce").dt.strftime("%Y-%m-%d")

w = w.copy()
w["date_norm"] = pd.to_datetime(w["date"], errors="coerce").dt.strftime("%Y-%m-%d")

ufo["place_key"] = ufo["location"].str.extract(r";s*([^\s;]+)")[0].str.strip().str.lower()
w["place_key"] = w["place"].astype(str).str.strip().str.lower()

merged = ufo.merge(
    w[["date_norm", "place_key", "cloud_cover", "precipitation", "temperature",
        "cloud_cover_avg", "precipitation_avg", "lat", "lon", "place"]],
    on=["date_norm", "place_key"],
    how="left",
    suffixes=("", "_weather")
)

print("after merge (date + place):", merged.shape)
print("rows without weather:", merged["cloud_cover"].isna().sum())
merged.head()

```

```

after merge (date + place): (137, 16)
rows without weather: 92
/tmp/ipykernel_6460/1636771225.py:2: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is consistent
  ufo["date_norm"] = pd.to_datetime(ufo["date"], errors="coerce").dt.strftime("%Y-%m-%d")

```

date

name

location

description

period

id

date

norm

place

key

cloud

cover

precipitation

temperature

cloud

cover

avg

precipitation

avg

lat

lon

place

```

df = merged
df["has_weather"] = df["cloud_cover"].notna()
df["cloudy"] = df["cloud_cover"] >= 50
df["cloudier_than_avg"] = df["cloud_cover"] > df["cloud_cover_avg"]
df["rainy"] = df["precipitation"] > 0
df["year"] = pd.to_datetime(df["date"], errors="coerce").dt.year

df[["date", "location", "cloud_cover", "cloud_cover_avg",
    "cloudy", "cloudier_than_avg", "year"]].head()

```

```

... /tmp/ipykernel_6460/2167135372.py:6: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `dateutil`. To ensure parsing is correct,
df["year"] = pd.to_datetime(df["date"], errors="coerce").dt.year

```

	date	location	cloud_cover	cloud_cover_avg	cloudy	cloudier_than_avg	year
0	c.1450BC	Africa Ancient Egypt; Jebel Barkal, Lower Egypt	NaN	NaN	False	False	NaN
1	218BC	Europe Roman Republic; Italia	NaN	NaN	False	False	NaN
2	76BC	Asia Roman Republic; Asia	NaN	NaN	False	False	NaN
3	73BC	Asia Roman Republic; Phrygia, Asia	NaN	NaN	False	False	NaN
4	65BC	Europe Roman Republic; Italia	NaN	NaN	False	False	NaN

[84]

✓ Os

```

daily = (df.groupby("date")
         .agg(sightings=("id", "count"),
             avg_cloud=("cloud_cover", "mean"),
             avg_temp=("temperature", "mean"))
         .reset_index())

df = df.merge(daily, on="date", how="left")

print(df[["date", "sightings", "avg_cloud", "avg_temp"]].head())
print("final shape:", df.shape)

```

▼

```

      date sightings avg_cloud avg_temp
0  c.1450BC         1      NaN      NaN
1    218BC         1      NaN      NaN
2    76BC         1      NaN      NaN
3    73BC         1      NaN      NaN
4    65BC         1      NaN      NaN
final shape: (137, 24)

```

|

s

```

print("Final dataframe:", df.shape)
print("gaps in main columns:\n",
      df[["cloud_cover", "temperature", "sightings"]].isna().sum())

print("\nCloudiness (describe):")
print(df["cloud_cover"].describe())

print("\nProportion of cloudy days (>=50%):", round(df["cloudy"].mean(), 3))

df.to_csv("ufo_clean.csv", index=False)
print("\nsaved: ufo_clean.csv", df.shape)

```

```

... Final dataframe: (137, 24)
gaps in main columns:
  cloud_cover    92
  temperature    92
  sightings       0
dtype: int64

Cloudiness (describe):
count    45.000000
mean     47.733333
std       33.627910
min        0.000000
25%       17.000000
50%       44.000000
75%       79.000000
max      100.000000
Name: cloud_cover, dtype: float64

```

Part 5. Analysis and Visualization

Input: ufo_clean.csv from Part 3 (ufo_raw.csv from Wikipedia merged with weather_data.csv from Open-Meteo by id). The goal is to check if UFO sightings happen on cloudier days than the normal weather in the same place and season.

Tools: Python, pandas, numpy, matplotlib, seaborn, scipy (Google Colab).

Approach

1. Loaded ufo_clean.csv (137 rows, 20 columns) and kept only the rows where weather was found (cloud_cover is not empty) – 58 rows. The other 79 sightings are before 1940, have no exact day or have only a region as location.
2. Added new columns: year and decade (from date_weather), cloud_diff (cloud cover on the sighting day minus normal cloud cover), cloudy (cloud cover 50% or more), rainy (precipitation above 0) and cloudier (the sighting day is cloudier than normal).
3. Calculated descriptive statistics with describe() for cloud_cover, cloud_cover_avg, precipitation, precipitation_avg and temperature, and the share of cloudy days, rainy days and days cloudier than normal.
4. Calculated correlations between the weather columns with corr().
5. Grouped the data by decade (groupby) and compared the average cloud cover on the sighting day with the normal one. Counted the events in each decade.
6. Compared the sighting day with the normal weather of the same event using a paired t-test (ttest_rel) and the Wilcoxon test. If the p-value is below 0.05, the difference is significant.
7. Built 4 plots with seaborn and matplotlib: histogram, scatter plot, boxplot and bar chart. Every plot has a title and axis labels.

Part 4. Analysis and Visualization

Step 1. Load data

We load ufo_clean.csv and keep only the rows where weather was found.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

df = pd.read_csv("ufo_clean.csv")
data = df[df["cloud_cover"].notnull()].copy()

print("all events:", df.shape)
print("events with weather:", data.shape)

all_events: (137, 20)
events with weather: (58, 20)
```

Step 2. New columns

We add year, decade and the difference between the sighting day and the normal cloud cover. We also mark cloudy days (cloud cover 50% or more), rainy days and days cloudier than normal.

```
data["year"] = pd.to_datetime(data["date_weather"]).dt.year
data["decade"] = data["year"] // 10 * 10
data["cloud_diff"] = data["cloud_cover"] - data["cloud_cover_avg"]
data["cloudy"] = data["cloud_cover"] >= 50
data["rainy"] = data["precipitation"] > 0
data["cloudier"] = data["cloud_cover"] > data["cloud_cover_avg"]

data.head()
```

	date	name	location	description	period	id	date_weather	cloud_cover	precipitation	temperature	...	cloudy	cloudier_than_avg	rainy	year	sightings	avg_cloud	avg_temp	decade	cloud_diff	cloudier
23	1942-02-24	Battle of Los Angeles	North America United States : Los Angeles, Cal.	Just months after the Japanese attack on Pearl...	1900-1949	23	1942-02-24	24.0	0.0	8.8	...	False	False	False	1942	1	24.0	8.8	1940	-18.0	False
26	1945-05-18	Ängelholm UFO memorial	Europe Sweden : Ängelholm, Kristianstads County	Swedish entrepreneur Gösta Carlsson, attribut...	1900-1949	26	1945-05-18	91.0	4.3	11.0	...	True	True	True	1945	1	91.0	11.0	1940	39.8	True
30	1947-07-07	Rhodes UFO photographs	North America United States : Phoenix, Arizona	Inventor and amateur astronomer William Albert...	1900-1949	30	1947-07-07	1.0	0.0	34.4	...	False	False	False	1947	1	1.0	34.4	1940	-15.8	False
31	1947-07-08	The Roswell Incident	North America United States : about 30 mi. nor...	Walter Haut, a United States Army Air Forces ...	1900-1949	31	1947-07-08	4.0	0.0	26.7	...	False	False	False	1947	1	4.0	26.7	1940	-21.8	False
36	1948-07-24	Chico-Whitted UFO encounter	North America United States : Montgomery, Alabama	Clarence Chico and John Whitted, American com...	1900-1949	36	1948-07-24	34.0	2.9	27.5	...	False	False	True	1948	1	34.0	27.5	1940	-28.4	False

5 rows × 23 columns

Step 3. Descriptive statistics

We calculate `describe()` for the weather columns and the share of cloudy days, rainy days and days cloudier than normal.

```
cols = ["cloud_cover", "cloud_cover_avg", "precipitation", "precipitation_avg", "temperature"]

print(data[cols].describe().round(1))
print()
print("share of cloudy days:", round(data["cloudy"].mean(), 2))
print("share of rainy days:", round(data["rainy"].mean(), 2))
print("share of days cloudier than normal:", round(data["cloudier"].mean(), 2))
```

	cloud_cover	cloud_cover_avg	precipitation	precipitation_avg	\
count	58.0	58.0	58.0	58.0	
mean	44.9	49.6	1.2	2.2	
std	33.6	19.3	2.4	2.1	
min	0.0	7.5	0.0	0.0	
25%	14.5	34.4	0.0	0.9	
50%	40.5	51.0	0.0	1.8	
75%	76.0	63.4	0.7	2.8	
max	100.0	86.4	8.5	12.9	

	temperature
count	58.0
mean	15.2
std	9.8
min	-10.2
25%	7.9
50%	15.7
75%	22.9
max	34.4

```
share of cloudy days: 0.45
share of rainy days: 0.36
share of days cloudier than normal: 0.43
```

Step 4. Correlations

We check how the weather columns are related to each other.

```
data[cols + ["cloud_diff"]].corr().round(2)
```

	cloud_cover	cloud_cover_avg	precipitation	precipitation_avg	temperature	cloud_diff
cloud_cover	1.00	0.57	0.51	0.26	-0.38	0.82
cloud_cover_avg	0.57	1.00	0.31	0.65	-0.39	-0.00
precipitation	0.51	0.31	1.00	0.26	-0.15	0.40
precipitation_avg	0.26	0.65	0.26	1.00	0.08	-0.14
temperature	-0.38	-0.39	-0.15	0.08	1.00	-0.20
cloud_diff	0.82	-0.00	0.40	-0.14	-0.20	1.00

Step 5. Groupby by decade

We calculate the average cloud cover on the sighting day and the normal cloud cover for each decade, and count the events in each decade.

```
print(data.groupby("decade")[["cloud_cover", "cloud_cover_avg"]].mean().round(1))
print()
print(data["decade"].value_counts().sort_index())
```

	cloud_cover	cloud_cover_avg
decade		
1940	30.8	39.0
1950	37.8	45.9
1960	46.8	51.3
1970	47.1	54.6
1980	58.0	52.4
1990	35.7	37.1
2000	50.4	50.6
2010	76.0	86.4
2020	91.0	74.5

decade	
1940	5
1950	15
1960	15
1970	9
1980	4
1990	3
2000	5
2010	1
2020	1

Name: count, dtype: int64

Step 6. Statistical test

Main question: is the sighting day cloudier than the normal weather in the same place and season? Both values are for the same event, so we use paired tests: t-test and Wilcoxon test. If the p-value is below 0.05, the difference is significant.

```
print("average on sighting day:", round(data["cloud_cover"].mean(), 1))
print("average normal:", round(data["cloud_cover_avg"].mean(), 1))

t_test = stats.ttest_rel(data["cloud_cover"], data["cloud_cover_avg"])
wilcoxon = stats.wilcoxon(data["cloud_cover"], data["cloud_cover_avg"])

print("t-test p-value:", round(t_test.pvalue, 3))
print("Wilcoxon p-value:", round(wilcoxon.pvalue, 3))

average on sighting day: 44.9
average normal: 49.6
t-test p-value: 0.196
Wilcoxon p-value: 0.226
```

Results

Sample: 58 sightings with weather data (of 137)

Cloud cover on the sighting day: mean 44.9%, median 40.5%. Normal cloud cover: mean 49.6%, median 51.0%

Cloudy days (50% or more): 45%. Days with precipitation: 36%. Days cloudier than normal: 43%

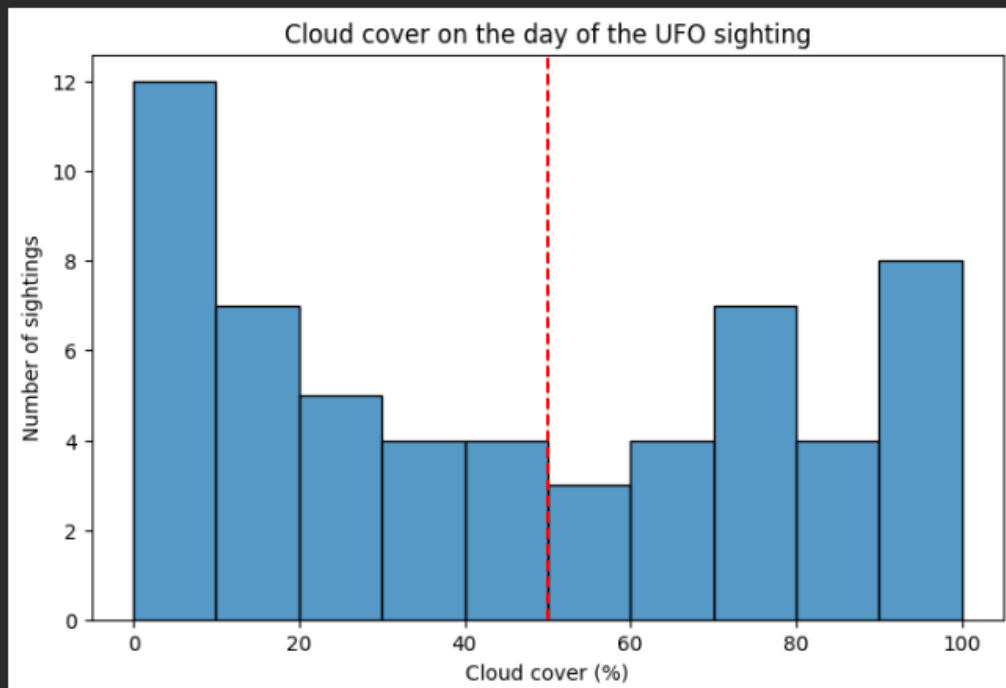
Paired t-test p-value = 0.196, Wilcoxon p-value = 0.226 – the difference is not significant

Correlations: cloud_cover and cloud_cover_avg $r = 0.57$, cloud_cover and precipitation $r = 0.51$, cloud_cover and temperature $r = -0.38$

By decade: most sightings are from the 1950s and 1960s (15 in each). From the 1940s to the 1970s the sighting day was a little clearer than normal. The 2010s and 2020s have only 1 sighting each, so these values are not reliable

Plot 1. Histogram: cloud cover on the sighting day

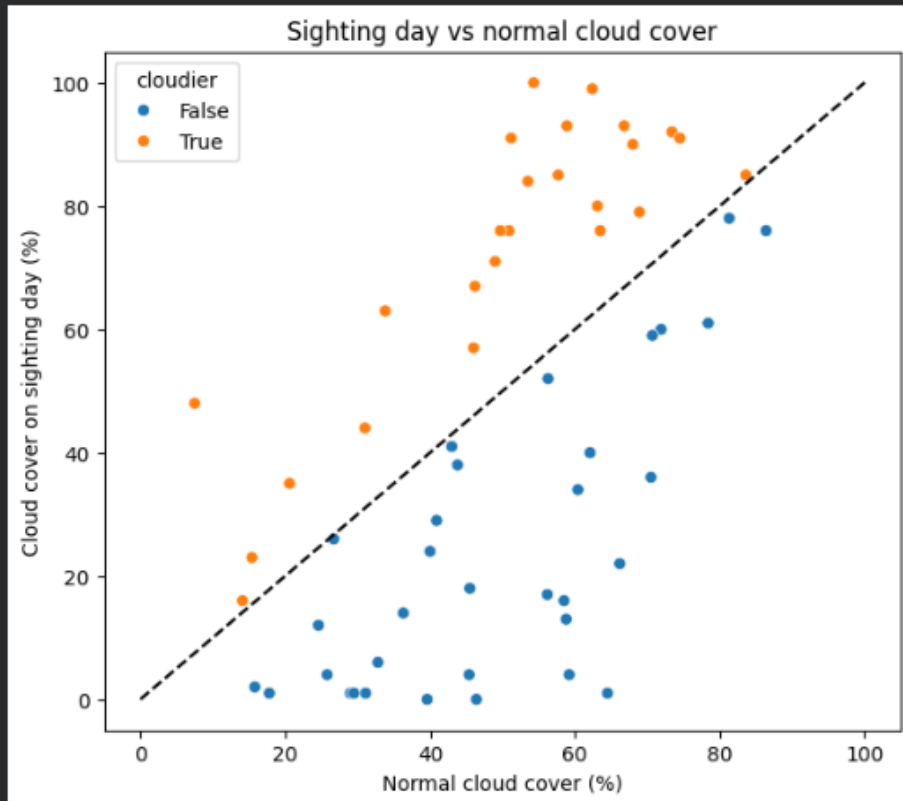
```
plt.figure(figsize=(8, 5))
sns.histplot(data["cloud_cover"], bins=10)
plt.axvline(50, color="red", linestyle="--")
plt.title("Cloud cover on the day of the UFO sighting")
plt.xlabel("Cloud cover (%)")
plt.ylabel("Number of sightings")
plt.savefig("plot1_hist.png", dpi=150)
plt.show()
```



Plot 2. Scatter: sighting day vs normal cloud cover

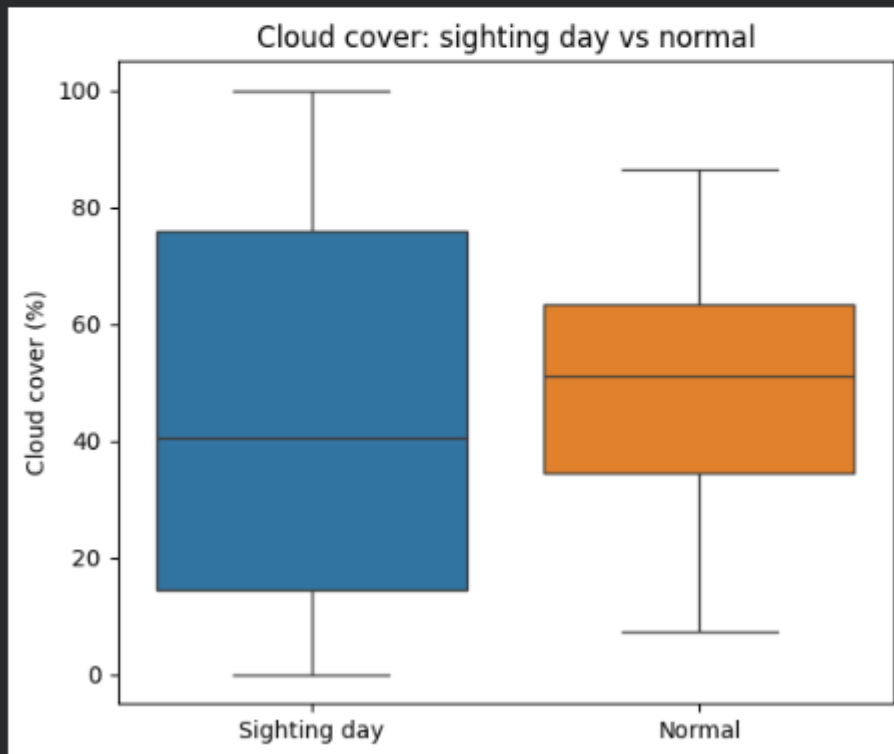
Points above the dashed line are cloudier than normal.

```
plt.figure(figsize=(7, 6))
sns.scatterplot(data=data, x="cloud_cover_avg", y="cloud_cover", hue="cloudier")
plt.plot([0, 100], [0, 100], color="black", linestyle="--")
plt.title("Sighting day vs normal cloud cover")
plt.xlabel("Normal cloud cover (%)")
plt.ylabel("Cloud cover on sighting day (%)")
plt.savefig("plot2_scatter.png", dpi=150)
plt.show()
```



Plot 3. Boxplot: sighting day vs normal cloud cover

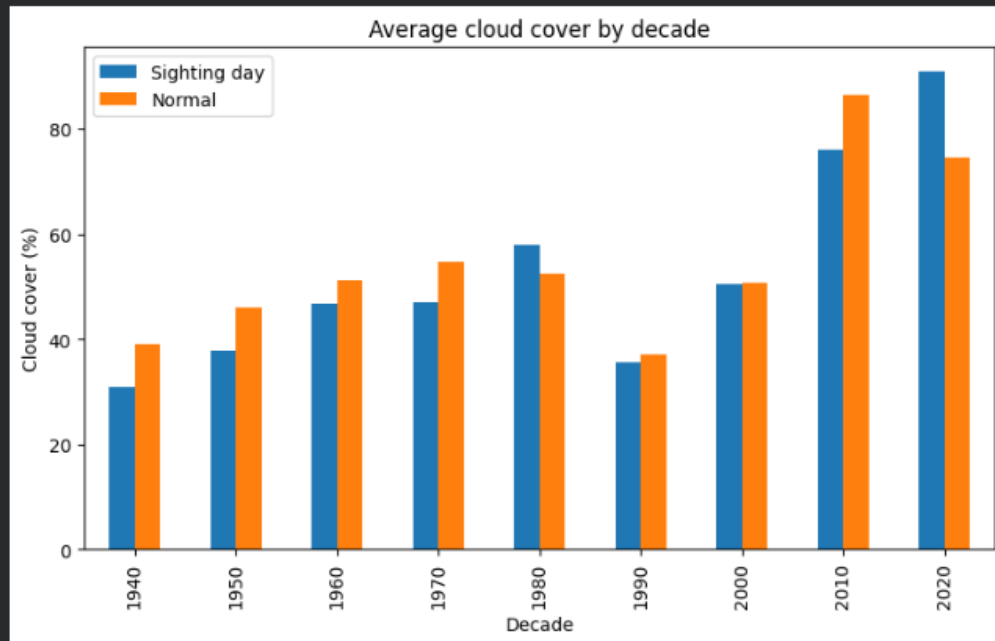
```
plt.figure(figsize=(6, 5))
sns.boxplot(data=data[["cloud_cover", "cloud_cover_avg"]])
plt.xticks([0, 1], ["Sighting day", "Normal"])
plt.title("Cloud cover: sighting day vs normal")
plt.xlabel("")
plt.ylabel("Cloud cover (%)")
plt.savefig("plot3_box.png", dpi=150)
plt.show()
```



Plot 4. Bar: average cloud cover by decade

```
by_decade = data.groupby("decade")[["cloud_cover", "cloud_cover_avg"]].mean()

by_decade.plot(kind="bar", figsize=(9, 5))
plt.legend(["Sighting day", "Normal"])
plt.title("Average cloud cover by decade")
plt.xlabel("Decade")
plt.ylabel("Cloud cover (%)")
plt.savefig("plot4_bar.png", dpi=150)
plt.show()
```



Conclusion: the data does not show that UFO sightings happen only on cloudy days. On the sighting day the cloud cover was even slightly lower than normal (44.9% vs 49.6%), and only 43% of the sightings were cloudier than normal. The sample is small (58 sightings) and consists of famous cases from Wikipedia, not of all sightings, so the result cannot be generalized.